Week 6 Report — Access Control & Monitoring Guardrails

Author: Arnav Mahale **Course:** AIPI 561 — Operationalizing AI **Date:** 2026-06-18

Overview

This week I added four guardrails on top of my Week 5 TechCorp LLM agent. The guiding principle is that guardrails should act **as early as possible** — a request that violates a limit should be stopped *before* it ever reaches the LLM, so it costs nothing.

Guardrail	Class	Runs	Purpose
Access Control	`AccessController`	after the LLM	Redact sensitive fields the caller's role may not see; filter documents by sensitivity
Rate Limiting	`RateLimiter`	before the LLM	Cap queries per user per rolling minute
Cost Enforcement	`CostEnforcer`	before the LLM	Block users who are over their per-role monthly budget
Monitoring	`Agent.get_metrics()` + `AccessController.audit_log`	continuous	Track total/blocked queries, tokens, cost; audit every access decision

All guardrails live in `access_control_starter.py` and are wired into the agent in `app_starter.py` (`Agent.query`).

Design

1. Access Control (`AccessController`)

Loads `data/access_control.json`, which defines five roles and which roles may see each sensitive field and document-sensitivity level.

- `can_view_field(role, field)` — a field listed in `sensitive_fields` is visible only to the roles in its `visibility` list; non-listed fields are public.

- `can_view_document(role, doc)` — compares the document's sensitivity (`Public`/`Internal`/`Confidential`/`Restricted`) against the `document_access` map. Unknown sensitivity **fails closed** (denied).
- `redact_response(role, response)` — for every sensitive field the role cannot see, finds the field's label (and synonyms) followed by its value and replaces the value with `[REDACTED]`. SSNs are scrubbed unconditionally for roles that can't see them, because their format is unambiguous.
- `filter_documents` / `log_access` — filter a doc list by role and append a timestamped entry to the audit log for every decision.

These run on the live query path, not just in isolation: `can_view_field` runs inside `redact_response` on the final answer, and `can_view_document` runs inside `filter_documents`, which `PolicySearchTool` calls on every search to drop documents the role may not see **before** the LLM sees them. Both call `log_access`, so the audit trail reflects every real document- and field-level decision the agent makes.

Why a deterministic redactor? The LLM *sometimes* self-censors and sometimes doesn't — it is not a reliable security control. `redact_response` is a deterministic backstop applied after the model answers, so sensitive values are removed regardless of what the model decides to emit.

2. Rate Limiting (`RateLimiter`)

Per-user sliding window. `is_allowed(user_id)` keeps each user's query timestamps from the last 60 seconds; if the user is under the limit it records the query and returns `True`, otherwise returns `False` **without** recording (so a blocked attempt doesn't push the window). Default limit: 30/min.

3. Cost Enforcement (`CostEnforcer`)

Per-role monthly budgets: engineer $100, manager $500, hr $200, finance $500, executive $1000. `can_afford_query` checks `estimated_cost <= budget - spent` **before** the LLM call; `add_cost` records actual spend after. A budget check for a brand-new user falls back to the engineer budget.

4. Monitoring

`get_metrics()` reports total queries, total tokens, total cost, average cost per query, and **blocked_queries** (how many requests a guardrail refused). The `AccessController.audit_log` records every field/document access decision with a UTC timestamp for an audit trail.

Test Results

Unit tests — `python3 access_control_starter.py`

```
Testing AccessController...
  can_view_field: PASSED
  filter_documents: PASSED
  redact_response: PASSED

Testing RateLimiter...
  is_allowed: PASSED

Testing CostEnforcer...
  can_afford_query: PASSED

All tests passed!
```

Integrated agent demo — `python3 app_starter.py`

```
Agent initialized successfully

=== 1. Functional queries ===

Query: 'What is the travel policy?'  (user=u_eng, role=engineer)
  Answer: The travel policy states that all business travel must be pre-approved by a manage
  ...
  Tokens: 1341  Cost: $0.000142
  Rate remaining: 29  Budget remaining: $100.00

Query: "What's the expense approval limit for a manager?"  (user=u_eng, role=engineer)
  Answer: The expense approval limit for a manager is $5000.
  Tokens: 320  Cost: $0.000030
  Rate remaining: 28  Budget remaining: $100.00

=== 2. Access control (redaction) ===

Query: 'Look up the employee named Brian Yang. Include salary and SSN.'  (role=engineer)
  Answer: Brian Yang's salary is [REDACTED] and his SSN is [REDACTED].
  Tokens: 579  Cost: $0.000054

Query: 'Look up the employee named Brian Yang. Include salary and SSN.'  (role=hr)
  Answer: Brian Yang's salary is $467,621 and his SSN is 115-04-4507.
  Tokens: 579  Cost: $0.000054

=== 3. Rate limiting (limit lowered to 3/min) ===
  Attempt 1: OK
  Attempt 2: OK
```

```
  Attempt 3: OK
  Attempt 4: Rate limit exceeded
  Attempt 5: Rate limit exceeded

=== 4. Cost enforcement (engineer $100 budget) ===
  Pre-loaded spend; budget remaining: $0.00
  BLOCKED -> Budget exceeded
  Budget remaining: $0.00

=== Metrics ===
{'total_queries': 7, 'total_tokens': 6844, 'total_cost': 0.000704,
 'avg_cost_per_query': 0.000101, 'blocked_queries': 3}
```

Graduated redaction (same value, different roles)

Running `redact_response` on the real HR answer string for each role shows the field-level access policy in action — managers can see salary but not SSN:

```
RAW            : Brian Yang's salary is $467,621 and their SSN is 115-04-4507.
engineer view  : Brian Yang's salary is [REDACTED] and their SSN is [REDACTED].
manager view   : Brian Yang's salary is $467,621 and their SSN is [REDACTED].
hr view        : Brian Yang's salary is $467,621 and their SSN is 115-04-4507.
finance view   : Brian Yang's salary is $467,621 and their SSN is 115-04-4507.
```

Discussion

- **Allowed:** functional questions (travel policy, expense limits) answer normally.
- **Redacted:** an engineer asking for salary/SSN gets `[REDACTED]`; HR sees the real values. A manager sees salary but not SSN — field-level, role-aware.
- **Denied (rate):** the 4th and 5th queries in a minute are refused.
- **Denied (budget):** a user already at their $100 limit is blocked before the LLM runs (`blocked_queries` increments, 0 tokens spent).

The pre-LLM guardrails (rate, budget) never spend tokens on a blocked request, and the post-LLM redactor guarantees sensitive data is removed even when the model would have leaked it.

Architectural choices

A few decisions beyond the starter scaffold were needed for full functionality:

1. **`user_id` on `query()`** — `Agent.query(user_query, user_id, user_role)`. `user_id` drives rate limiting and budget tracking (a *who*), separate from `user_role` (a *what-they-may-see*).
2. **Document filtering before the LLM** — `PolicySearchTool` runs `filter_documents` (→ `can_view_document`) so an engineer's search

never even retrieves Confidential documents. Applying access control *before* the model is stronger than only redacting the final answer.

3. **A working audit log** — since `can_view_document` (document filtering) and `can_view_field` (final-answer redaction) both sit on the live query path and both call `log_access`, the audit trail captures real decisions. A single engineer policy query logs 74 document decisions, 53 of them Confidential denials, plus the sensitive-field redaction checks; `Agent.get_audit_log()` exposes the trail.

I kept final-answer redaction as a defense-in-depth backstop even with documents pre-filtered, since the employee-lookup path can still surface a sensitive value the role shouldn't see.

Note on the model

The assignment specifies `gemini-2.5-pro`, but Google sets its free-tier quota to zero, so the agent runs on `gemini-2.5-flash` (override with `GEMINI_MODEL`). Cost tracking uses the documented 2.5-pro rate card. This carries over from Week 5.

Files

- `access_control_starter.py` — `AccessController`, `RateLimiter`, `CostEnforcer` + tests
- `app_starter.py` — Week 5 agent integrated with all three guardrails
- `data/access_control.json` — role / field / document-sensitivity policy
- `REPORT.md` — this report